

UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE MECÁNICA ELÉCTRICA,
ELECTRÓNICA Y SISTEMAS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

INFORME TÉCNICO:

Unidad I: Análisis, Modelado, Arquitectura y Desarrollo
Avanzado Orientado a Objetos

CURSO: PROGRAMACIÓN ORIENTADA A OBJETOS II

DOCENTE: ZANABRIA GALVEZ ALDO HERNAN

ALUMNOS: YANA MENDOZA CARLOS BENEDICTO

REPOSITORIO: https://github.com/Benyamen-droid/PROYECTO_POO2

Semestre: III

GRUPO: B

PUNO – PERÚ
2026 – I

Índice

1. PARTE 1: ARQUITECTURA Y MODELADO UML	2
1.1. 1.1 Diagrama de Casos de Uso	2
1.2. 1.2 Diagrama de Clases Estructural	2
2. PARTE 2: IMPLEMENTACIÓN EN C++17	2
2.1. 2.1 Estructura de Archivos	2
2.2. 2.2 Polimorfismo Dinámico — Despacho Virtual	3
2.3. 2.3 Gestión de Memoria con Smart Pointers (RAII)	3
2.4. 2.4 Verificación de Compilación	4
3. PARTE 3: IMPLEMENTACIÓN EN PYTHON 3.11+	4
3.1. 3.1 Módulo abc — Clases Abstractas Nativas	4
3.2. 3.2 Desafío Avanzado — Persistencia Bidireccional (Opción A)	4
3.3. 3.3 Salida de Ejecución Verificada	5
4. PARTE 4: MATRIZ COMPARATIVA INTER-LENGUAJES	5
5. PARTE 5: CUESTIONARIO DE INVESTIGACIÓN TÉCNICA	5
5.1. Pregunta 1: Mecanismos de Abstracción	5
5.2. Pregunta 2: Impacto del Polimorfismo en Arquitecturas de Software	7
5.3. Pregunta 3: Relevancia del Modelado UML en el Desarrollo Ágil	8
6. PARTE 6 (DESAFÍO AVANZADO): MOTOR DE PERSISTENCIA — OPCIÓN A	8
6.1. Descripción de la Implementación	8
6.2. Volcado a Disco (Serialización)	9
6.3. Hidratación de Objetos con Control de Excepciones	9
7. CONCLUSIONES	9
Referencias Bibliográficas	10

1. PARTE 1: ARQUITECTURA Y MODELADO UML

1.1. 1.1 Diagrama de Casos de Uso

El sistema SmartLib v2.0 modela los siguientes actores y relaciones normativas:

- Actores: Bibliotecario (actor principal), Estudiante y Profesor (generalizados desde Usuario General).
- Caso de uso Realizar Préstamo incluye («include») Consultar Disponibilidad de Catálogo, garantizando que la disponibilidad del material sea verificada antes de cualquier transacción.
- Realizar Préstamo se extiende («extend») hacia Aplicar Restricción por Límite cuando el usuario superó su cuota (3 para Estudiante, 10 para Profesor).
- Devolver Libro se extiende («extend») hacia Calcular y Registrar Penalidad cuando se detecta retraso en la devolución, disparando el cálculo diferenciado de mora.

1.2. 1.2 Diagrama de Clases Estructural

La arquitectura de clases implementa las siguientes jerarquías e interconectividades:

Clases Abstractas Puras

- Persona: Base para todos los usuarios. Declara `mostrarPerfil()` y `calcularTarifaMora()` como métodos virtuales puros.
- Encapsula `uid` y `nombre` como privados, con `libros_prestados` como protegido.
- MaterialBibliotecario: Base para el inventario. Declara `mostrarDetalle()` y `tipo()` como métodos abstractos. Gestiona el estado transaccional mediante el enum `EstadoMaterial`.

Herencia Directa

- Estudiante hereda de Persona con `maxLibros=3` y tarifa de mora \$1.50/día.
- Profesor hereda de Persona con `maxLibros=10` y tarifa de mora \$0.50/día.
- LibroFisico hereda de MaterialBibliotecario añadiendo `pasillo`, `estante` y `numeroCopias`.
- LibroDigital hereda de MaterialBibliotecario añadiendo `urlAcceso`, `tamanoMB` y `formato`.

Relaciones de Composición y Agregación

2. PARTE 2: IMPLEMENTACIÓN EN C++17

2.1. 2.1 Estructura de Archivos

- `Persona.h` — Clase abstracta pura base de usuarios

Relación	Tipo	Cardinalidad
Biblioteca → MaterialBibliotecario	Composición ()	1 a 0..*
Biblioteca → Persona	Composición ()	1 a 0..*
Biblioteca → Prestamo	Agregación ()	1 a *
Prestamo → Persona + Material	Asociación directa	Entidad de asociación

Cuadro 1: Relaciones de Clases

- `Estudiante.h` — Especialización con máx. 3 préstamos, mora \$1.50/día
- `Profesor.h` — Especialización con máx. 10 préstamos, mora \$0.50/día
- `MaterialBibliotecario.h` — Clase abstracta base del inventario
- `LibroFisico.h` — Material con pasillo y estante
- `LibroDigital.h` — Material con URL y tamaño en MB
- `Prestamo.h` — Entidad de asociación Usuario ↔ Material
- `Biblioteca.h` — Contenedor principal con STL + smart pointers
- `main.cpp` — Flujo completo de transacciones simuladas
- `CMakeLists.txt` — Configuración de compilación automatizada

2.2. 2.2 Polimorfismo Dinámico — Despacho Virtual

El sistema implementa polimorfismo dinámico mediante tablas virtuales (vtable). Las clases base declaran métodos con virtual puro (= 0), y las subclases los sobrescriben con override. Al iterar una colección de punteros base, el compilador resuelve el método correcto en tiempo de ejecución:

```
// Colección polimórfica de tipo base
vector<shared_ptr<Persona>> usuarios = {
    make_shared<Estudiante>(...), // Polimorfismo
    make_shared<Profesor>(...)    // Dinámico
};

for (const auto& u : usuarios) {
    u->mostrarPerfil();           // Despacho en runtime
    u->calcularTarifaMora(5);     // Resuelto por vtable
}
```

2.3. 2.3 Gestión de Memoria con Smart Pointers (RAII)

Se utiliza `std::shared_ptr` para garantizar la liberación automática de memoria al destruir el contenedor, eliminando fugas de memoria sin necesidad de delete manual:

```
// shared_ptr: memoria liberada automáticamente
auto estudiante = make_shared<Estudiante>(...);
biblioteca.registrarUsuario(estudiante);

// El objeto se destruye solo al salir del scope
// No se requiere: delete estudiante;
```

2.4. 2.4 Verificación de Compilación

El código compila limpiamente con las flags requeridas sin ninguna advertencia:

```
$g++ -std=c++17 -Wall -Wextra -o smartlib main.cpp$ ./smartlib
```

```
SMARTLIB v2.0 - Sistema de Biblioteca  
Universidad Tecnológica | C++ 17
```

```
[SALIDA VERIFICADA]: 0 errores, 0 advertencias.  
5 transacciones procesadas. Penalidades calculadas.  
Reporte de auditoría persistido en: auditoria_smartlib.txt
```

3. PARTE 3: IMPLEMENTACIÓN EN PYTHON

3.11+

3.1. 3.1 Módulo abc — Clases Abstractas Nativas

Python no tiene clases abstractas nativas de forma predeterminada; se usa el módulo abc con el decorador @abstractmethod para simular el mismo contrato que los métodos virtuales puros de C++:

```
from abc import ABC, abstractmethod
from typing import List, Optional

class Persona(ABC):
    """Clase abstracta pura - interfaz común para usuarios."""

    def __init__(self, uid: str, nombre: str, max_libros: int):
        self.__uid = uid          # Name mangling: privado fuerte
        self.__nombre = nombre
        self._libros_prestados = 0 # Protegido

    @property
    def uid(self) -> str:
        return self.__uid        # Encapsulación via descriptor

    @abstractmethod
    def mostrar_perfil(self) -> None: ...

    @abstractmethod
    def calcular_tarifa_mora(self, dias: int) -> float: ...
```

3.2. 3.2 Desafío Avanzado — Persistencia Bidireccional (Opción A)

Se implementó el motor de persistencia en JSON y CSV con hidratación de objetos completa:

```
# Serialización: volcar estado a JSON
def exportar_usuarios_json(self, ruta='usuarios.json'):
```

```

    datos = [u.to_dict() for u in self.__lista_usuarios]
    with open(ruta, 'w', encoding='utf-8') as f:
        json.dump(datos, f, ensure_ascii=False, indent=2)

# Hidratación: reconstruir objetos desde JSON
def hidratar_usuarios_json(self, ruta='usuarios.json'):
    try:
        with open(ruta) as f:
            datos = json.load(f)
        for d in datos:
            if d['tipo'] == 'Estudiante':
                u = Estudiante(d['uid'], d['nombre'], ...)
    except json.JSONDecodeError:
        print('[ERROR] Archivo corrupto o mal formado.')

```

3.3. 3.3 Salida de Ejecución Verificada

```

$ python3 smartlib.py

-> Usuario 'Carlos_Mendoza' indexado correctamente en Python.
Préstamo PRE-1: 'Design_Patterns' → Carlos Mendoza
[RESTRICCIÓN] Carlos Mendoza alcanzó el límite de 3 préstamos.
MORA APLICADA: $7.50 (5 días de retraso).
[PERSISTENCIA] Usuarios exportados a 'usuarios.json'.
[HIDRATACIÓN] 4 usuarios reconstruidos desde 'usuarios.json'.

[RESULTADO]: 0 excepciones no controladas. PEP 8 cumplido.

```

4. PARTE 4: MATRIZ COMPARATIVA INTER-LENGUAJE

Basada en la experiencia empírica de codificar el mismo sistema en ambos ecosistemas:

5. PARTE 5: CUESTIONARIO DE INVESTIGACIÓN TÉCNICA

5.1. Pregunta 1: Mecanismos de Abstracción

¿Qué diferencia crítica existe entre una clase abstracta y una interfaz pura?

Una clase abstracta y una interfaz pura difieren fundamentalmente en su propósito arquitectónico y en lo que pueden contener:

- Las clases abstractas permiten definir estados con variables de instancia (atributos propios) y lógica de métodos concretos (implementaciones parciales compartidas).
- Son apropiadas cuando existe comportamiento común reutilizable entre subclases, tal como los métodos `registrar_nuevo_prestamo()` y `registrar_devolucion()` en la clase `Persona` de `SmartLib`, que son idénticos para `Estudiante` y `Profesor` y por lo tanto no se declaran abstractos.
- Las interfaces puras (o protocolos), por su parte, imponen únicamente contratos de comportamiento abstractos sin ningún estado.

Dimensión Analítica	C++17	Python 3.11+
Velocidad de Ejecución	Ultra alta. Compila a lenguaje máquina nativo. Polimorfismo vía vtable con latencia mínima en tiempo de enlace.	Moderada. Interpreta bytecode sobre CPython. Duck Typing y búsquedas en <code>__dict__</code> generan mayor sobrecarga en runtime.
Curva de Aprendizaje	Pronunciada. Exige gestión de punteros, ciclo de vida de objetos, enlazado de headers y directivas del compilador.	Suave. Sintaxis limpia y cercana al pseudocódigo. Sin gestión de memoria explícita ni declaraciones de tipo obligatorias.
Manejo de Memoria	Semiautomática. Uso de RAII con <code>std::unique_ptr</code> y <code>std::shared_ptr</code> . Sin operadores <code>new/delete</code> explícitos en código moderno.	Totalmente automática. Garbage Collector basado en conteo de referencias con detector de ciclos generacionales.
Rigurosidad Sintáctica	Tipado estático. Verificación en tiempo de compilación. Errores de firma de métodos detendrán la generación del ejecutable.	Tipado dinámico. Las variables adoptan el tipo del objeto asignado. Errores de tipo se detectan en runtime.
Velocidad de Desarrollo	Lenta. Requiere CMakeLists, archivos .h separados, constructores de copia explícitos y destructores virtuales.	Muy rápida. Estructuras de datos nativas (listas, dicts, sets), sin boilerplate técnico, ecosistema de testing ágil.
Abstracción Pura	Métodos virtuales puros (= 0). Clase sin atributos y con solo virtuales puros simula una interfaz.	Módulo abc con <code>@abstractmethod</code> . Adicionalmente typing.Protocol para contratos estructurales sin herencia.
Persistencia de Datos	Manual con <code><fstream></code> . Mayor verbosidad pero control total sobre formatos binarios y planos.	Módulos json y csv nativos. Serialización con <code>to_dict()</code> e hidratación con control de excepciones.

Cuadro 2: Matriz Comparativa

- Son la herramienta del arquitecto para definir lo que un objeto debe saber hacer, no cómo lo hace ni qué datos almacena.
- En ingeniería de software, las interfaces favorecen el Principio de Segregación de Interfaces (ISP) de SOLID, evitando que las clases dependan de contratos que no necesitan.

Implementación en C++17 En C++, una interfaz pura se simula declarando una clase con únicamente métodos virtuales puros (= 0) y sin atributos de instancia. El compilador prohíbe instanciar dicha clase directamente. Ejemplo en SmartLib:

```
class Persona { // Clase Abstracta (tiene atributos + lógica)
protected:
    string uid, nombre;
```

```

    int librosPrestadosActuales;
public:
    bool registrarNuevoPrestamo() { ... } // Concreto
    virtual void mostrarPerfil() const = 0; // Puro
    virtual double calcularTarifaMora(int) const = 0; // Puro
};

```

Implementación en Python 3.11+ Python utiliza el módulo abc (Abstract Base Classes). La distinción entre clase abstracta y protocolo se logra de dos formas:

```

from abc import ABC, abstractmethod
from typing import Protocol

# 1. Clase abstracta (puede tener estado y lógica concreta)
class Persona(ABC):
    def __init__(self, uid, nombre, max_libros):
        self.__uid = uid # Tiene estado
    def registrar_devolucion(self): ... # Lógica concreta
    @abstractmethod
    def mostrar_perfil(self): ... # Contrato puro

# 2. Protocolo estructural (pura interfaz, sin herencia)
class UsuarioProtocol(Protocol):
    def mostrar_perfil(self) -> None: ...
    def calcular_tarifa_mora(self, dias: int) -> float: ...

```

5.2. Pregunta 2: Impacto del Polimorfismo en Arquitecturas de Software

¿Cuáles son las ventajas estratégicas del polimorfismo dinámico en sistemas empresariales?

El polimorfismo dinámico es uno de los pilares arquitectónicos más poderosos en el desarrollo de sistemas de gran envergadura. Sus ventajas estratégicas se articulan alrededor de dos conceptos fundamentales:

Desacoplamiento de Software El polimorfismo permite que los módulos del sistema interactúen a través de contratos (interfaces o clases abstractas) en lugar de implementaciones concretas. En SmartLib, la clase Biblioteca opera únicamente con referencias de tipo `Persona*` o `shared_ptr<Persona>`, sin conocer si el objeto real es un Estudiante o un Profesor. Esto reduce dramáticamente las dependencias entre módulos, facilitando las pruebas unitarias (mocking), el mantenimiento y la evolución del sistema.

Principio Abierto/Cerrado (OCP de SOLID) Formulado por Bertrand Meyer y popularizado por Robert C. Martin, el principio Abierto/Cerrado establece que un módulo debe estar abierto para extensión pero cerrado para modificación. El polimorfismo es el mecanismo que hace esto posible. Ejemplo concreto en SmartLib: si la universidad decide agregar un nuevo tipo de usuario, Alumno de Posgrado, con un límite de 7 préstamos y una tarifa de mora de \$1.00/día, basta con crear una nueva subclase sin modificar una sola línea del motor transaccional (`Biblioteca.realizarPrestamo`, `Biblioteca.registrarDevolucion`) ni de las clases existentes:

```

class AlumnoPosgrado : public Persona {
public:
    AlumnoPosgrado(string uid, string nombre)
        : Persona(uid, nombre, 7) {}

```



```

void mostrarPerfil() const override { ... }

double calcularTarifaMora(int dias) const override {
    return (dias > 0) ? (dias * 1.00) : 0.0;
}
};

// El motor transaccional existente NO cambia
biblioteca.registrarUsuario(make_shared<AlumnoPosgrado>(...));
// Funciona de inmediato. Riesgo de regresión = 0

```

Esta propiedad reduce drásticamente el riesgo de introducir bugs de regresión al añadir funcionalidad, ya que el código probado y funcionando no se toca. En sistemas con miles de usuarios concurrentes y SLAs críticos de disponibilidad, este beneficio es invaluable.

5.3. Pregunta 3: Relevancia del Modelado UML en el Desarrollo Ágil

¿Por qué el modelado UML sigue siendo crucial en entornos ágiles modernos?

Existe un mito en algunos equipos ágiles de que el modelado UML es una práctica de metodologías waterfall incompatible con Scrum o Kanban. Esta visión es incorrecta y costosa. El UML no es burocracia; es comunicación arquitectónica de precisión quirúrgica.

Valor de la Comunicación Arquitectónica Universal Un diagrama de clases UML es un lenguaje compartido entre arquitectos de software, desarrolladores backend, líderes técnicos, analistas de negocio y stakeholders técnicos sin importar el lenguaje de programación que usen. En SmartLib, el diagrama de clases comunica instantáneamente la jerarquía Persona → Estudiante/Profesor, la composición con Biblioteca y la entidad de asociación Prestamo a cualquier ingeniero del equipo, sin necesidad de leer miles de líneas de código.

Diseño antes de Codificar — Prevención de Errores Estructurales Los errores de diseño estructural (relaciones de herencia incorrectas, violaciones de encapsulamiento, dependencias circulares) detectados en el diagrama UML cuestan minutos de corrección. Los mismos errores detectados en producción cuestan semanas de refactorización, regresiones y potenciales incidentes de disponibilidad. En el contexto ágil, el modelado UML funciona como el plano de un arquitecto en construcción: no se dibuja cada tornillo, sino la estructura portante del edificio. Los diagramas de casos de uso comunican el valor de negocio al Product Owner en el Sprint Planning; los diagramas de clases guían las decisiones técnicas de los desarrolladores en el diseño de APIs y contratos de módulo. En definitiva, el modelado UML no ralentiza el desarrollo ágil: lo protege de errores de alto costo que el código ágil, por su naturaleza iterativa, suele descubrir tardíamente.

6. PARTE 6 (DESAFÍO AVANZADO): MOTOR DE PERSISTENCIA — OPCIÓN A

6.1. Descripción de la Implementación

Se implementó el módulo de persistencia bidireccional completo en Python, eligiendo la Opción A: serialización a JSON (usuarios) y CSV (catálogo), con hidratación de objetos

con control de excepciones.

6.2. Volcado a Disco (Serialización)

Cada clase de dominio implementa el método `to_dict()` que retorna su representación en diccionario serializable, siguiendo el patrón Memento:

```
# usuarios.json generado automáticamente:
[
  {
    "uid": "E001",
    "nombre": "Carlos Mendoza",
    "tipo": "Estudiante",
    "codigo_estudiante": "20241045",
    "carrera": "Ingeniería de Software"
  },
  ...
]
```

6.3. Hidratación de Objetos con Control de Excepciones

La reconstrucción de objetos desde JSON incluye manejo de errores para archivos inexistentes, corruptos o con formato inválido, garantizando tolerancia a fallos básicos según los requisitos del sistema:

```
def hidratar_usuarios_json(self, ruta='usuarios.json'):
    try:
        with open(ruta, 'r', encoding='utf-8') as f:
            datos = json.load(f)
        for d in datos:
            if d['tipo'] == 'Estudiante':
                u = Estudiante(d['uid'], d['nombre'], ...)
            elif d['tipo'] == 'Profesor':
                u = Profesor(d['uid'], d['nombre'], ...)
            self.registrar_usuario(u)
    except FileNotFoundError:
        print('[ERROR] Archivo no encontrado.')
    except json.JSONDecodeError:
        print('[ERROR] Archivo corrupto o mal formado.')
```

El catálogo se exporta a `catalogo.csv` con todos los campos de cada material (tipo, identificador, título, autor, año, categoría y campos específicos de cada subclase), permitiendo su consulta en herramientas de hoja de cálculo externas.

7. CONCLUSIONES

- La implementación del sistema SmartLib v2.0 demostró que la Programación Orientada a Objetos es un paradigma que trasciende los lenguajes de programación: el mismo modelo conceptual (clases abstractas, herencia, polimorfismo dinámico, composición y agregación) se traduce fielmente tanto en C++17 como en Python 3.11+, con diferencias únicamente en la sintaxis y en los mecanismos de bajo nivel.

- El polimorfismo dinámico a través de colecciones de punteros base (`shared_ptr<Persona>` en C++ y `List[Persona]` en Python) permitió procesar usuarios heterogéneos (Estudiante y Profesor) con un único ciclo iterador, demostrando en la práctica el Principio Abierto/Cerrado y el beneficio del desacoplamiento.
- C++17 con smart pointers (`shared_ptr/unique_ptr`) ofrece gestión de memoria determinista y rendimiento máximo a costa de una mayor complejidad sintáctica.
- Python 3.11+ prioriza la velocidad de desarrollo y legibilidad mediante su Garbage Collector automático y el módulo `abc` para contratos abstractos.
- El modelado UML previo al desarrollo permitió identificar y resolver las relaciones entre clases (composición vs. agregación, entidad de asociación Prestamo) antes de escribir una sola línea de código, validando su utilidad como plano arquitectónico en metodologías ágiles.
- El Desafío Avanzado (Opción A) de persistencia bidireccional añade valor operacional real al sistema: el estado de la biblioteca sobrevive a los reinicios del programa mediante la serialización JSON/CSV y la hidratación robusta con tolerancia a fallos.

Referencias Bibliográficas (APA 7)

- Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3rd ed.). Prentice Hall.
- Lippman, S. B., Lajoie, J., & Moo, B. E. (2023). *C++ Primer* (6th ed.). Addison-Wesley.
- Phillips, D. (2018). *Python Object-Oriented Programming* (3rd ed.). Packt Publishing.
- Python Software Foundation. (2026). *Python Language Documentation* (v3.11+). <https://docs.python.org/3/>